The Gate Signal

Implicit Coordination by an Observable Route,
under Epistemic and Classical Planning, with Two Robots and No Radio

Haniel Ulises

Epistemic Robotics

13 October 2026

Two robots must deliver one load each into two bays, one load per bay. An order board tells $r1$ which bay its load goes to; $r2$ must deliver into the other, and only into a bay it knows $r1$ will not use. There is no radio. From its booth $r2$ sees two gates and, of $r1$'s routes, nothing else; $r1$ can reach its bay by a back route nobody sees or by a longer route through the gate of that bay. The goal asks no one to know anything. We prove that every policy of the domain routes $r1$ through its gate, since the precondition of $r2$'s delivery can be met in no other way, and that Aletheia's policy of depth 7 does so; that $r1$ planning classically alone never does; and that a joint classical plan presupposes communication. Given $r2$ a second source of knowledge, a look into a bay, the planner returns a policy of the same depth that uses it, and only durations can tell the two apart. In Gazebo, over 20 paired seeds, the policy delivered correctly in 20 of 20 runs with $r2$ knowing the order 23.5 s after the start; its expected makespan was 79.0 s and its worst 85.0 s, against 113.2 s and 176.2 s for the strongest correct classical team, which checks a bay; the optimistic classical team was right in 10 of 20.

Contents

1 Scope **2**

2 Environment **2**

3 Domain **3**

4 The policy **3**

5 Classical planning **4**

6 Steps and seconds **4**

7 Paired evaluation over 20 seeds **5**

8 Recorded runs **5**

9 Changes on five defects **6**

10 Limitations **6**

1 Scope

Implicit coordination is coordination without messages: each agent acts on what it can observe of the others, and the others act so that what it observes is enough [3]. The interesting case is the one in which an agent changes what it does only so that another will know something. That is a choice no classical planner makes, since classical planning has no representation of what another agent will know, and it is a choice an epistemic planner can be shown to make for exactly the reason that it is needed.

The scenario is built so that the knowledge one robot needs can come from one place: the sight of the other in a gate. Going through the gate costs the other robot a longer route. The comparison is with classical teams at their strongest: each robot plans with POPF [4] over a model with every route timed from the roadmap it drives, and the robot without the order is given the best strategy there is for it in this hall.

2 Environment

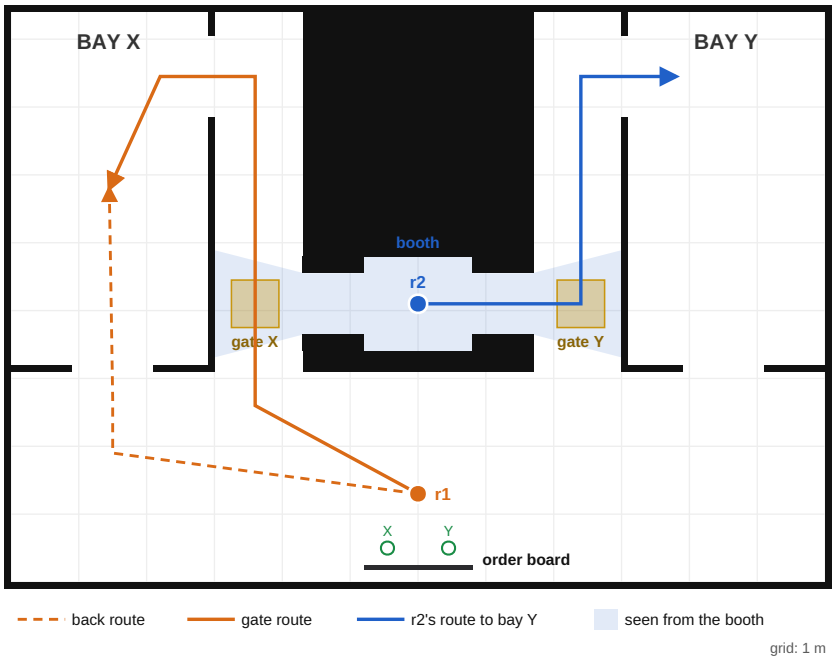


Figure 1: The hall, 12 by 8.4 m. Bay X and bay Y are walled rooms with doors onto the south corridor; between them, a block of racking with two passages, one beside each bay, each opening by a side door into its bay, and $r2$'s booth, joined to each passage by a slit at the height of the gate. Shaded: what $r2$'s laser reaches from its station. Orange: $r1$'s back route to bay X (dashed, 8.44 m) and its route through gate X (10.79 m). Blue: $r2$'s route to bay Y.

Both robots are TurtleBot3 Burgers with their lasers' range set to 8 m, each with its ROS interfaces in its own namespace; the simulator's odometry is the world pose. Before any simulator run, `tools/check_views.py` casts $r2$'s 360 rays from its station against the walls and checks the assumptions the domain makes about what $r2$ sees: $r1$ standing in either gate is hit by five rays; no point of either back route, of either bay's inside, of the south corridor or of $r1$'s start is hit by any. Seed s orders bay X when s is odd and bay Y when it is even, and perturbs both starts by up to 0.08 m and 8°.

Every observation is made by a robot's own laser. The read of the order counts returns in the two slots of the board; the watch counts returns of $r1$ in the two gate regions, through the

slits; each is reported to the executor as the event of the action's model that occurred, which the epistemic state then applies.

3 Domain

Two agents, $r1$ and $r2$. Atoms: *dest-x*, the order names bay X; for each agent i, *carrier*(i), *at-start*(i), *at-gate-x*(i), *at-gate-y*(i), *in-x*(i), *in-y*(i), *at-station*(i) and *delivered*(i). Actions are event models [1], planned over as in [2]:

action	type	precondition	observed by $r2$
read-order($r1$)	semi-private sensing	$r1$ at its start	that it happened
to-gate($r1$)	semi-private ontic	at start, Kw_{r1} *dest-x*	that it happened
back-route($r1$)	semi-private ontic	at start, Kw_{r1} *dest-x*	that it happened
enter-bay($r1$)	semi-private ontic	$r1$ in a gate	that it happened
watch($r2, r1$)	semi-private sensing	$r2$ at station, $r1$ in a gate	which gate
go-x($r2$), go-y($r2$)	public ontic	$r2$ at station	fully
drop-carrier($r1$)	semi-private ontic	$r1$ in the bay of its order	that it happened
drop-other-x($r2$)	public ontic	$r2$ in bay X, $K_{r2}\neg$*dest-x*	fully
drop-other-y($r2$)	public ontic	$r2$ in bay Y, K_{r2} *dest-x*	fully

Each semi-private action has two designated events, one per bay, with preconditions that differ on *dest-x* or on where $r1$ stands; the actor observes which occurred and every other agent only that one did. For an ontic action this needs a type the library shipped with ePlanSys does not have, two designated events with effects; the domain declares it, *semi-private-ontic*, in a library of its own, `gate.epddl`. The initial model has two worlds, both designated, differing only on *dest-x*; $r1$ is the carrier and stands at its start, $r2$ at its station, and neither can tell the worlds apart. The frame is S5. No action sends anything. The goal is *delivered*($r1$) $\wedge$ *delivered*($r2$), of modal depth 0.

4 The policy

Proposition 1 (The gate is necessary)**.** In every policy that reaches the goal from the initial model, every branch contains to-gate($r1$) followed by watch($r2, r1$) before $r2$'s drop, and no branch contains back-route($r1$).

Proof. *delivered*($r2$) is made true only by drop-other-x($r2$) and drop-other-y($r2$), since drop-carrier needs its actor to be the carrier. Their preconditions are $K_{r2}\neg$*dest-x* and K_{r2} *dest-x*. In the initial model $r2$'s relation joins the two worlds, which differ on *dest-x*, so neither holds; one comes to hold only once $r2$'s relation separates the image of the *dest-x*-world from that of the other. In a product update, $r2$'s relation separates two worlds only through the events applied to them: two updated worlds are related for $r2$ exactly when their old worlds were and their events are related for $r2$. Every action $r2$ observes partially relates all its events for $r2$, and every action with a single event applies the same event to both worlds; neither kind separates them. The two-event actions $r2$ observes fully are those with $r2$ as actor. read-order($r2$), to-gate($r2$), back-route($r2$) and drop-carrier($r2$) need $r2$ to be the carrier, and enter-bay($r2$) and watch($r2, r2$) need $r2$ in a gate, which only to-gate($r2$) brings about; none is ever applicable. That leaves watch($r2, r1$), whose events need *at-gate-x*($r1$) and *at-gate-y*($r1$); in no reachable state do both hold, and with neither it is inapplicable. Only to-gate($r1$) makes either true, and it needs *at-start*($r1$), which back-route($r1$) deletes and nothing restores. So every branch reaching *delivered*($r2$) has to-gate($r1$), then watch($r2, r1$), and no earlier back-route($r1$); and after to-gate, $r1$ is no longer at its start, so no later one either. □

Proposition 2 (The policy)**.** The policy

read-order($r1$);
  e-order-x : to-gate($r1$); watch($r2, r1$); enter-bay($r1$); drop-carrier($r1$);
    go-y($r2$); drop-other-y($r2$)
  e-order-y : to-gate($r1$); watch($r2, r1$); enter-bay($r1$); drop-carrier($r1$);
    go-x($r2$); drop-other-x($r2$)

is applicable in both worlds, reaches the goal in both, and has depth 7.

Proof. After read-order, $r1$'s relation separates the worlds and Kw_{r1} *dest-x* holds in both, so to-gate is applicable; in the *dest-x*-world its event e-gate-x puts $r1$ in gate X. watch($r2, r1$) then applies e-seen-x, which $r2$ observes fully, and $r2$'s relation separates the worlds: K_{r2} *dest-x* holds in the image of the *dest-x*-world. enter-bay puts $r1$ in bay X, drop-carrier delivers it there, go-y puts $r2$ in bay Y and drop-other-y's precondition holds. The other branch is symmetric. □

Aletheia returns this policy by AO* in 215 expansions, and its validator checks both leaves. The executor runs the actions the policy leaves unordered between the two robots together: enter-bay($r1$) with $r2$'s drive, and the two drops.

5 Classical planning

Proposition 3 ($r1$ alone)**.** In $r1$'s classical model, which has its goal, its two routes timed from the roadmap, 50.9 s by the back route and 68.0 s through the gate, and nothing about $r2$, every plan that goes through the gate is longer than one that does not, and no cost-optimal plan signals.

Proof. Replace to_gate and enter_bay in such a plan by back_route to the same bay: the result reaches the same state, since nothing else in the model refers to the gates, in less time. Over the batch, POPF took the back route in all 40 classical runs. □

Proposition 4 (Both together)**.** A classical planner given both robots and the order returns a joint plan in which $r1$ takes the back route and $r2$ drives directly to the other bay; the part of it that is $r2$'s is executable only by a controller that knows the order.

Proof. With the order known, a gate costs time and gains nothing, as for $r1$ alone. $r2$'s part of the plan names the bay, and $r2$'s controller, receiving no message, cannot know which. □

A classical joint plan is a plan for a team that communicates. A plan each robot can execute from what it knows is what implicit coordination asks for [3], and finding one means planning over what each robot will know at each step.

Without the order, the classical $r2$ runs one of two formulations, each over POPF in a PlanSys2 stack of its own. *Optimistic*: it plans its delivery as if both bays were free, and POPF sends it to bay X. *Cautious*: it drives into bay X and looks over it from the side door, repeatedly, until it sees $r1$ there or until 50 s after the start, the latest $r1$ could arrive (the slowest classical $r1$ measured was in its bay at 44.0 s); then it unloads in bay X, or crosses to bay Y through the booth and unloads there. The cautious $r2$ is always right, and it is the strongest classical $r2$ in this hall: without the signal it cannot know the order before $r1$ is in a bay, a bay can be looked over only from its side door, and checking either first is the same by symmetry.

6 Steps and seconds

The domain gives $r2$ one way to learn the order. The variant `gate-signal-look.epddl` gives it a second: look-x($r2, r1$), semi-private sensing from inside bay X, with events for $r1$ there and not

there, and cross($r2$) from bay X to bay Y. Aletheia then returns, by AO* in 466 expansions:

read-order($r1$); back-route($r1$); drop-carrier($r1$); go-x($r2$); look-x($r2, r1$);
  occupied : cross($r2$); drop-other-y($r2$)
  free : drop-other-x($r2$)

This is the cautious classical $r2$'s strategy, found from the knowledge precondition, and its depth is 7, the gate policy's. AO* minimises the actions on the longest branch and knows no durations, so it has no reason to prefer either; Section 7 measures them.

The first version of the variant let any agent look. The planner returned a policy of depth 6 in which $r1$, in bay X, looked over bay X: $r2$ observed the look happen, and since a look in bay X can happen only with the looker in bay X, $r2$ knew where $r1$ was. The planner was correct about the model; the model let $r2$ learn from an occurrence it could not see in the hall. The look was restricted to the robot that does not carry the order.

7 Paired evaluation over 20 seeds

Each seed is run under the three formulations side by side, every mission starting at the same simulated instant. A run is correct when both loads are delivered, $r1$'s into the bay of its order and $r2$'s into the other. The makespan is the later of the two deliveries in simulated seconds from the start; distances come from the odometry.

	correct	median, X	median, Y	expected	worst	$r2$ drove
epistemic policy	20/20	79.6 s	78.3 s	79.0 s	85.0 s	6.7 m
cautious classical	20/20	166.2 s	59.4 s	113.2 s	176.2 s	13.4 m
optimistic classical	10/20	55.0 s	55.2 s	55.6 s	63.1 s	6.7 m

Table 1: The batch. Medians by the bay the order names; expected is the mean of the two orders' means, the makespan when either order is as likely; $r2$ drove: mean distance.

The policy delivered correctly in every run, and $r2$ knew the order 23.5 s after the start, at worst 24.1 s. The gate costs $r1$ time: it was in its bay at a median of 56.0 s against 43.4 s by the back route, and drove 10.5 m against 8.3 m. When the order names bay X, the policy was faster than the cautious team in 10 of 10 seeds (exact sign test, $p = 0.002$), by 87.3 s (Hodges–Lehmann): the cautious $r2$ found $r1$ in bay X and crossed the hall, 10 times in all. When it names bay Y, the cautious team was faster in 10 of 10 ($p = 0.002$), by 18.4 s: its first guess was right, and then it needs no signal. With either order as likely, the expected makespan was 79.0 s for the policy and 113.2 s for the cautious team. The optimistic team was the fastest, 55.6 s, and delivered both loads into bay X in 10 of 20 runs, every one whose order names bay X (McNemar's exact test against the policy, $p = 0.0020$). Planning took a median of 0.15 s for the policy.

8 Recorded runs

Three takes on seed 1, whose order names bay X, recorded one at a time with the launch unchanged and headless: a Gazebo client on a virtual display, the world held paused until it had drawn, and the camera cut on the executors' own actions. The policy, the cautious team and the optimistic team; the film ends with the policy and the cautious team side by side by simulated time since the start.

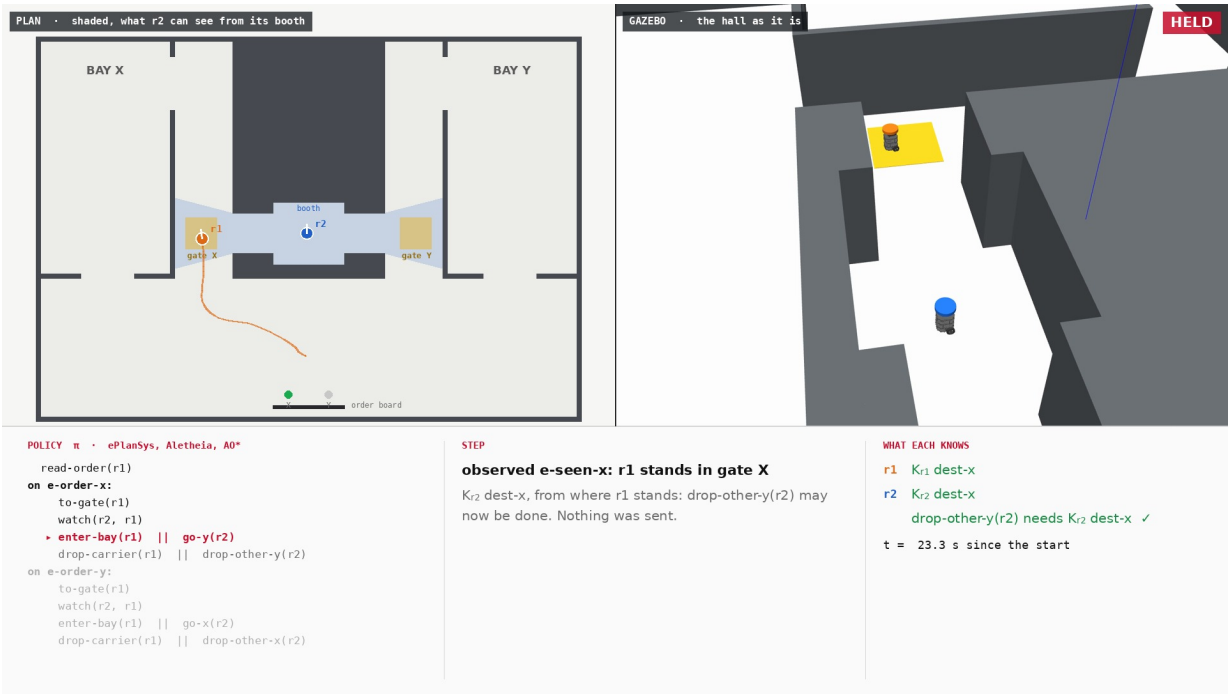


Figure 2: The film as $r2$ watches, held. Left: the hall from above, drawn from the recorded poses, with what $r2$ can see shaded; $r1$ (orange) stands in gate X. Right: Gazebo from above the booth, $r1$ in view through the slit. Under both: the policy with watch done, the observation e-seen-x, and what each robot knows: K_{r1} *dest-x* from the board, K_{r2} *dest-x* from the gate.

9 Changes on five defects

A model that told $r2$ too much. The first domain made $r1$'s drop and its entry into a bay public, with the bay in their preconditions; $r2$, seeing them happen, learned the order without any gate, and the planner used that. Both are semi-private.

A library type without conditions. *semi-private-ontic*, declared without `:conditions`, made plank exit with a segmentation fault while grounding. Each of its events declares that it has postconditions.

Every designated event's knowledge, required at once. Before it dispatches an action, the executor checks the modal part of every designated event's precondition. With to-gate's events requiring K_{r1} *dest-x* and $K_{r1}\neg$*dest-x*, it refused the first drive. Both events require Kw_{r1} *dest-x*, which with the order true in the world is the same, and the planner returns the same policy.

A parameter file the middleware could not read. One list of libraries, given to two nodes, was written as a YAML alias, and rcl refused the file. Each node gets its own copy.

Robots that crawled. In passages 1 m wide the controller's slow zone, 0.45 m from any obstacle, covered the whole width, and the first runs drove at 0.08 m/s. The passages, slits and doors were widened, the view from the booth checked again, and the batch run on the widened hall.

10 Limitations

- Durations. Whether a signal is worth its detour depends on durations the planner does not see; with a second source of knowledge for $r2$ the choice between two policies of equal depth is not the planner's.

- A hand-written convention. “Carriers go through their gate” would do as well here. The planner derives it from the precondition of $r2$’s delivery and from what $r2$ can see.
- One planner for both robots. The policy is computed once, from a model both robots share, and executed by one executor; each robot acts only on what it observes, but the robots do not plan separately.
- Unloads dispatched as a pair. The executor dispatches independent actions in groups, a group starting when the previous one has ended, so $r1$ unloads with $r2$; the makespan is $r2$’s either way.
- Decision-theoretic planners. A decentralized POMDP solver could find the same signal; the comparison here is with classical planning.

References

[1] A. Baltag, L. S. Moss and S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *Proceedings of TARK VII*, 43–56, 1998.

[2] T. Bolander and M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[3] T. Engesser, T. Bolander, R. Mattmüller and B. Nebel. Cooperative epistemic multi-agent planning for implicit coordination. *Proceedings of the Ninth Workshop on Methods for Modalities*, EPTCS 243, 75–90, 2017.

[4] A. J. Coles, A. I. Coles, M. Fox and D. Long. Forward-chaining partial-order planning. *Proceedings of ICAPS*, 42–49, 2010.